

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Computación

Redes

1966

UNIVERSIDAD

GUATEMALA



PROYECTO I

CHATBOT UTILIZANDO UN PROTOCOLO EXISTENTE

DEL VALLE DE

André Emilio Pivaral López – 23574

Excelencia que trasciende

Catedrático: Kevin Antonio Velásquez Aguilar

Sección: 10

Nueva Guatemala de la Asunción, 21 de septiembre de 2026

Índice

1. Resumen	1
2. Introducción	1
2.1. JSON-RPC 2.0	2
2.2. Ciclo de vida de una sesión	2
3. Arquitectura de la solución	3
4. Caso de uso: cadena de farmacias	6
5. Funcionamiento del anfitrión	6
5.1. Conexión con el modelo y contexto de sesión	6
5.2. Servidores oficiales: Filesystem y Git	7
5.3. Servidor propio en modo local	8
5.4. Bitácora de interacciones	8
6. Especificación del servidor propio	11
6.1. Identidad y capacidades	11
6.2. Métodos implementados	12
6.3. Herramientas	13
6.4. Endpoints y códigos HTTP	14
6.5. Errores de protocolo y errores de negocio	14
7. Despliegue del servidor remoto	15
8. Análisis de la comunicación	18
8.1. Metodología	18
8.2. Resolución del nombre del servidor	19
8.3. Establecimiento de la conexión	19
8.4. Mensajes del protocolo	21

8.5. Sesión y encabezados	22
8.6. Contenido JSON-RPC	23
8.7. Conversación completa	23
8.8. Estadística de la conversación	24
9. Análisis por capas	26
9.1. Capa de enlace	26
9.2. Capa de red	26
9.3. Capa de transporte	27
9.4. Seguridad y aplicación	27
9.5. Contraste con la captura de loopback	27
10. Conclusiones	29
10.1. Dificultades y soluciones	29
10.2. Lecciones aprendidas	30

1. Resumen

Este informe documenta la implementación de un chatbot de consola que actúa como anfitrión del protocolo Model Context Protocol, conocido como MCP. El anfitrión conversa con un modelo de lenguaje a través de su API, mantiene el contexto de la sesión y coordina tres servidores: los servidores oficiales de Filesystem y Git, y un servidor propio construido sobre un caso de uso a nivel de industria, la atención al cliente de una cadena de farmacias. El formato y el intercambio de mensajes se implementaron de forma manual sobre JSON-RPC 2.0, sin utilizar librerías ni SDKs que implementen el protocolo.

El servidor propio se ejecuta en dos variantes que comparten el mismo núcleo: una local sobre el transporte stdio y una remota sobre el transporte Streamable HTTP, desplegada en la plataforma Render. La comunicación entre el anfitrión y el servidor remoto se capturó con Wireshark, se descifró mediante el registro de claves de sesión TLS y se analizó por capas, contrastándola con una captura del mismo servidor sobre la interfaz de loopback.

2. Introducción

Un modelo de lenguaje solo conoce la información con la que fue entrenado y no puede, por sí mismo, consultar una base de datos, leer un archivo o registrar una compra. Model Context Protocol estandariza la manera en que una aplicación le entrega al modelo el acceso a esas capacidades externas, de modo que una herramienta desarrollada una sola vez pueda ser utilizada por cualquier anfitrión compatible [1].

El protocolo define tres actores. El servidor expone herramientas y ejecuta el trabajo real. El cliente mantiene una sesión con un único servidor, negocia la versión del protocolo y descubre las herramientas disponibles. El anfitrión es la aplicación con la que interactúa el usuario; coordina uno o varios clientes y decide, turno por turno, cuándo invocar una herramienta.

MCP es un protocolo de capa de aplicación que no define su propio formato de mensajes, sino que lo delega en JSON-RPC 2.0 [2], y este a su vez viaja sobre el transporte que se elija. La relación es análoga a la de HTTP con TCP: MCP define qué mensajes existen y en qué orden se intercambian, JSON-RPC define cómo se estructura cada mensaje, y el transporte define por dónde viajan.

2.1. JSON-RPC 2.0

JSON-RPC es un mecanismo de llamada a procedimiento remoto que define únicamente tres tipos de mensaje. Una solicitud contiene los campos `jsonrpc`, `method`, `params` y un identificador `id`. Una respuesta devuelve ese mismo identificador junto con `result` o con `error`, nunca ambos. Una notificación es igual a una solicitud pero sin identificador, por lo que el otro extremo no debe responderla.

El identificador cumple una función equivalente a la de un número de secuencia: permite emparejar cada respuesta con su solicitud aun cuando lleguen intercaladas con otros mensajes. La especificación también estandariza los códigos de error de la Tabla 1.

Código	Significado
−32700	Error de parseo: el mensaje no es JSON válido
−32600	Solicitud inválida: el objeto no cumple la estructura de JSON-RPC
−32601	Método no encontrado
−32602	Parámetros inválidos
−32603	Error interno del servidor

Tabla 1: Códigos de error estándar de JSON-RPC 2.0

2.2. Ciclo de vida de una sesión

Toda sesión inicia con un saludo de tres mensajes. El cliente envía la solicitud `initialize`, en la que propone la versión del protocolo y declara sus capacidades; el servidor responde con la versión que acepta, sus propias capacidades y su identificación. El cliente confirma con la notificación `notifications/initialized`. Finalmente, el cliente solicita `tools/list` y recibe el catálogo de herramientas, cada una con un esquema JSON de sus parámetros. A partir de ese momento, el mensaje que se repite es `tools/call`.

El descubrimiento mediante `tools/list` es lo que hace posible la interoperabilidad: el servidor se describe a sí mismo, de modo que un servidor escrito por terceros funciona sin modificar el código del cliente.

3. Arquitectura de la solución

La solución se implementó en Python con la librería estándar, sin dependencias externas. La API del modelo de lenguaje se consume con `urllib` y no con el SDK del proveedor, de manera que tanto el tráfico hacia el modelo como el tráfico hacia los servidores quedan bajo control directo del código. La Tabla 2 resume la responsabilidad de cada módulo.

Archivo	Responsabilidad
<code>main.py</code>	Punto de entrada, encabezado institucional y flujo de menús
<code>chatbot.py</code>	Anfitrión: historial de la conversación y ciclo de invocación de herramientas
<code>configuracion.py</code>	Lectura del archivo de entorno y mensaje de sistema
<code>consola.py</code>	Formato de las salidas de consola y lectura de entradas
<code>bitacora.py</code>	Registro de solicitudes, respuestas, notificaciones y errores
<code>servidores.json</code>	Definición de los servidores que consume el anfitrión
<code>protocolo/jsonrpc.py</code>	Construcción y validación manual de mensajes JSON-RPC 2.0
<code>protocolo/transporte_stdio.py</code>	Transporte stdio con hilos lectores dedicados
<code>protocolo/transporte_http.py</code>	Transporte Streamable HTTP con manejo de sesión
<code>protocolo/cliente_mcp.py</code>	Ciclo de vida del protocolo
<code>protocolo/administrador.py</code>	Coordinación de clientes y consolidación del catálogo
<code>llm/cliente_anthropic.py</code>	Consumo de la API de mensajes del modelo
<code>servidor/dominio.py</code>	Reglas de negocio de la cadena de farmacias
<code>servidor/nucleo_mcp.py</code>	Lado servidor del protocolo y especificación de herramientas
<code>servidor/servidor_local.py</code>	Servidor propio sobre stdio
<code>servidor/servidor_remoto.py</code>	Servidor propio sobre Streamable HTTP
<code>servidor/Dockerfile</code>	Imagen de contenedor para el despliegue remoto

Tabla 2: Módulos de la solución

El Fragmento 1 muestra el saludo inicial tal como lo ejecuta el cliente. Cada llamada a `solicitar` construye el mensaje con un identificador incremental, lo registra en la bitácora, lo entrega al transporte y espera la respuesta que trae ese mismo identificador.

Fragmento 1: Saludo inicial en `cliente_mcp.py`

```
resultado = self.solicitar("initialize", {
    "protocolVersion": self.version_protocolo,
    "capabilities": {"roots": {"listChanged": False}, "sampling": {}},
    "clientInfo": {"name": self.nombre_cliente, "version": self.version_cliente
},
})
self.notificar("notifications/initialized")
self.listar_herramientas()
```

El anfitrión nunca delega la ejecución en el modelo. El modelo responde con bloques `tool_use` que contienen el nombre de la herramienta, sus argumentos y un identificador; el anfitrión ejecuta la herramienta real y devuelve el resultado en un bloque `tool_result` con ese mismo identificador, dentro del historial de la conversación. El Fragmento 2 muestra ese ciclo de forma simplificada. Como el historial se envía completo en cada petición, incluidos los bloques de invocación y de resultado, en esa misma lista queda resuelto el mantenimiento del contexto de la sesión.

Fragmento 2: Ciclo de invocación en `chatbot.py`, versión simplificada

```
usos = self.llm.extraer_usos(respuesta)
if not usos:
    break
resultados = []
for uso in usos:
    salida, es_error = self.administrador.ejecutar(uso["name"], uso.get("input"
) or {})
    resultados.append({"type": "tool_result", "tool_use_id": uso["id"],
                        "content": salida, "is_error": es_error})
self.mensajes.append({"role": "user", "content": resultados})
```

Como tres servidores conectados al mismo tiempo pueden exponer herramientas con nombres iguales, el administrador prefija cada nombre con la clave del servidor. La herramienta `crear_pedido` del servidor local se expone al modelo como `farmacia__crear_pedido` y la del servidor remoto como `remota__crear_pedido`; el prefijo también le indica al administrador hacia qué cliente enrutar cada invocación.

4. Caso de uso: cadena de farmacias

El servidor propio implementa el escenario propuesto en el enunciado: un chatbot que atiende a los clientes de una farmacia, los orienta según sus síntomas y les permite comprar medicamentos. El escenario se simula con cinco clientes con alergias y condiciones crónicas registradas, cuatro sucursales con inventario propio, y un catálogo de doce medicamentos identificados por su principio activo, de los cuales ocho son de venta libre y cuatro requieren receta.

Las reglas de seguridad farmacéutica se imponen del lado del servidor y no en el mensaje de sistema del modelo:

- Ante un signo de alarma, como dolor en el pecho o dificultad para respirar, la evaluación de síntomas devuelve una urgencia de atención inmediata y ninguna sugerencia de producto.
- En menores de doce años se deriva a un profesional, ya que la dosificación depende del peso.
- Un medicamento que requiere receta no se despacha si el cliente no tiene una receta vigente registrada.
- Un principio activo que choca con una alergia declarada bloquea el pedido.
- Toda respuesta clínica incluye un aviso que indica que la orientación no sustituye la evaluación de un profesional de salud.

Esta separación es el sentido mismo del protocolo: la herramienta es dueña de las reglas y el modelo únicamente orquesta las llamadas. Aunque el modelo intentara despachar un antibiótico sin receta, el servidor rechazaría la operación.

5. Funcionamiento del anfitrión

5.1. Conexión con el modelo y contexto de sesión

La Figura 1 muestra dos consultas consecutivas que no requieren herramientas. La segunda pregunta no menciona a Alan Turing y aun así se resuelve correctamente, ya que el anfitrión reenvía el historial completo en cada petición. El modelo, además, señala que la consulta está fuera del alcance de la farmacia, en cumplimiento del mensaje de sistema.

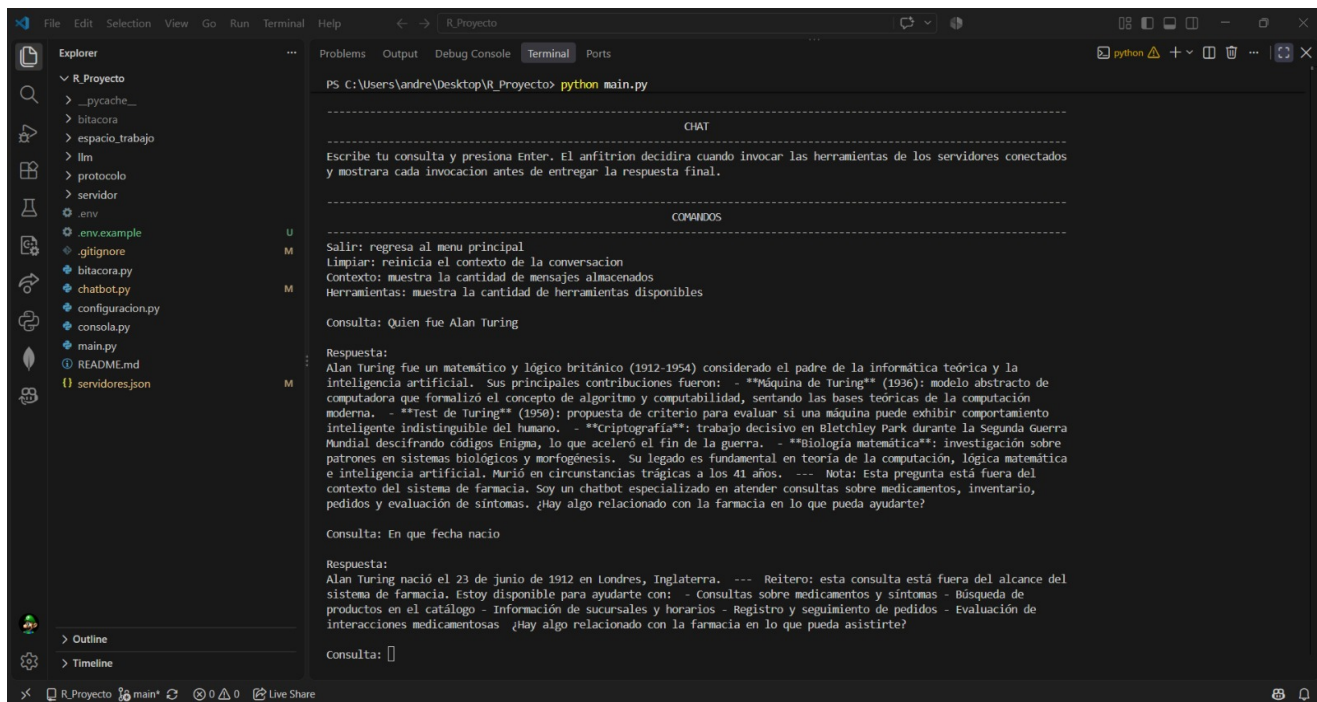


Figura 1: Respuesta a una pregunta general y seguimiento resuelto por contexto

5.2. Servidores oficiales: Filesystem y Git

Con los servidores de Filesystem y Git conectados, se solicitó al chatbot crear un archivo, agregarlo al repositorio y registrar un commit. La Figura 2 muestra las cuatro invocaciones que decidió el modelo.

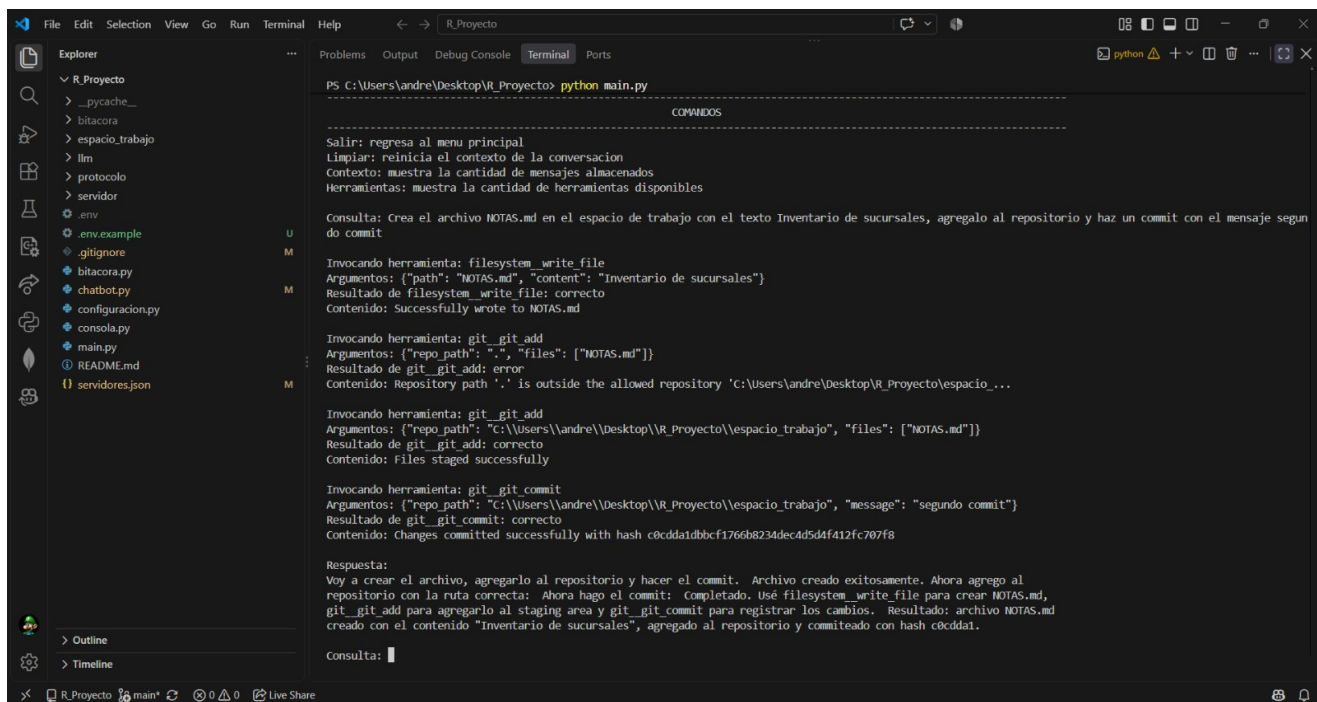


Figura 2: Creación de un archivo y commit mediante los servidores oficiales

La primera invocación de `git__git_add` falló porque el modelo indicó la ruta relativa del directorio actual, que queda fuera del repositorio permitido. El servidor devolvió el error como resultado de la herramienta, el modelo lo leyó y reintentó con la ruta absoluta del espacio de trabajo. Este comportamiento ilustra la diferencia entre un error de protocolo, que interrumpe el intercambio, y un error de herramienta, que se entrega como información para que el modelo corrija su propia acción. La Figura 3 confirma, fuera del programa, que el commit quedó registrado.

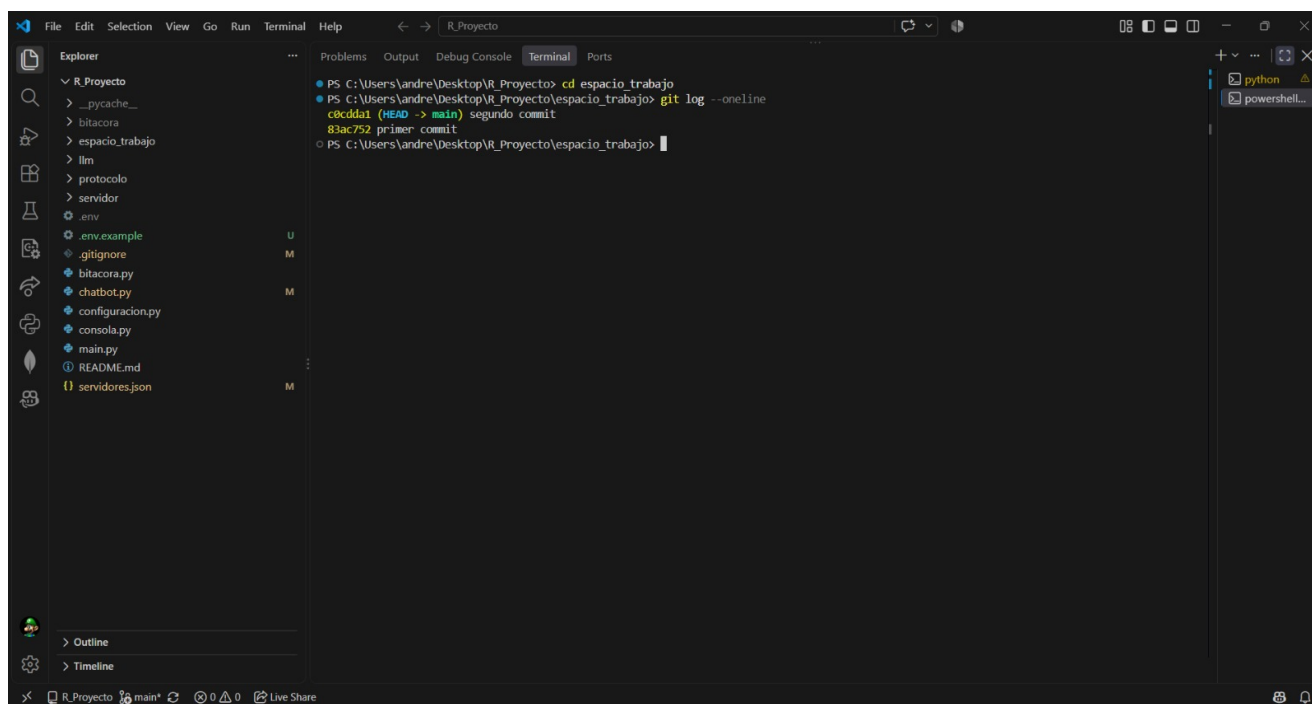


Figura 3: Historial del repositorio de trabajo con el commit generado por el chatbot

5.3. Servidor propio en modo local

La Figura 4 muestra una solicitud de compra de amoxicilina por parte de un cliente con alergia declarada a la penicilina. El modelo consultó primero la ficha del cliente y luego el catálogo, identificó que el principio activo es incompatible con la alergia y que el producto requiere una receta que el cliente no tiene registrada, y rechazó la solicitud ofreciendo alternativas.

5.4. Bitácora de interacciones

Cada mensaje enviado y recibido queda registrado con su marca de tiempo, el servidor involucrado, el método y el identificador. La Figura 5 muestra el resumen de

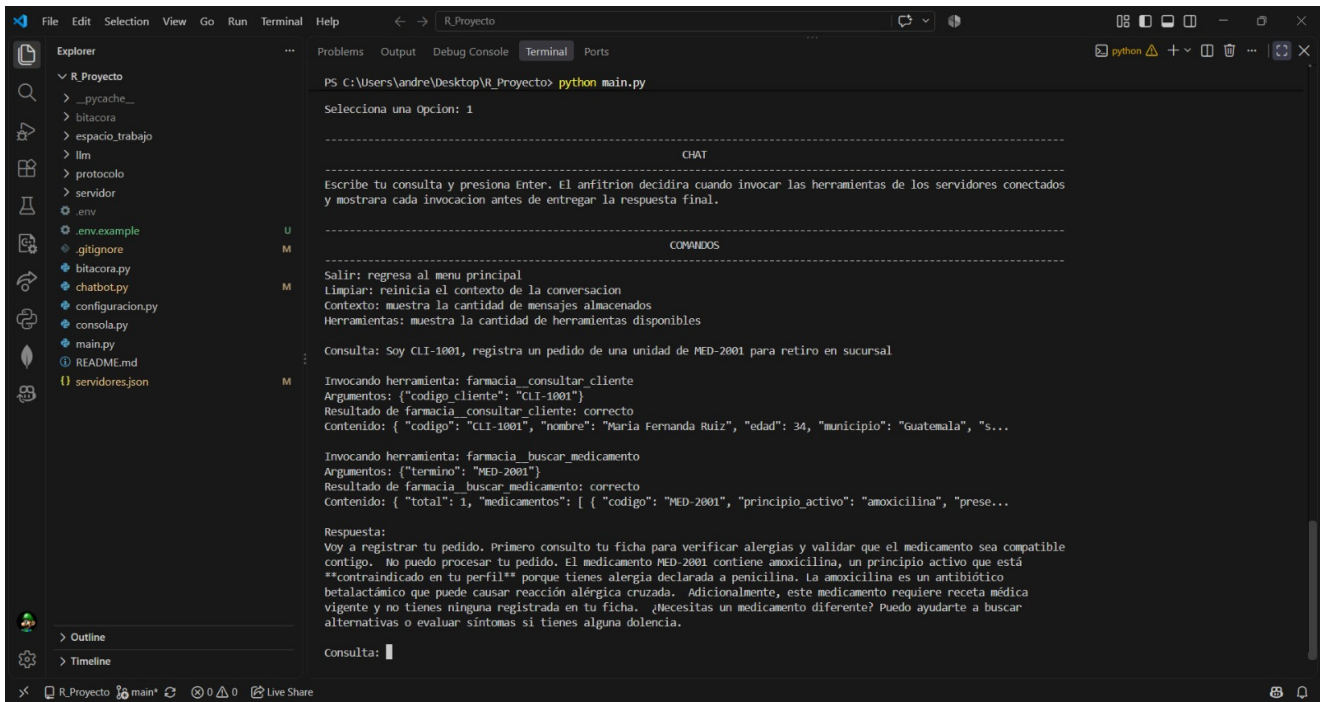


Figura 4: Consulta al servidor propio en modo local sobre stdio

una sesión con cuarenta interacciones, en el que se distingue con claridad la secuencia initialize, notifications/initialized, tools/list y tools/call de cada servidor.

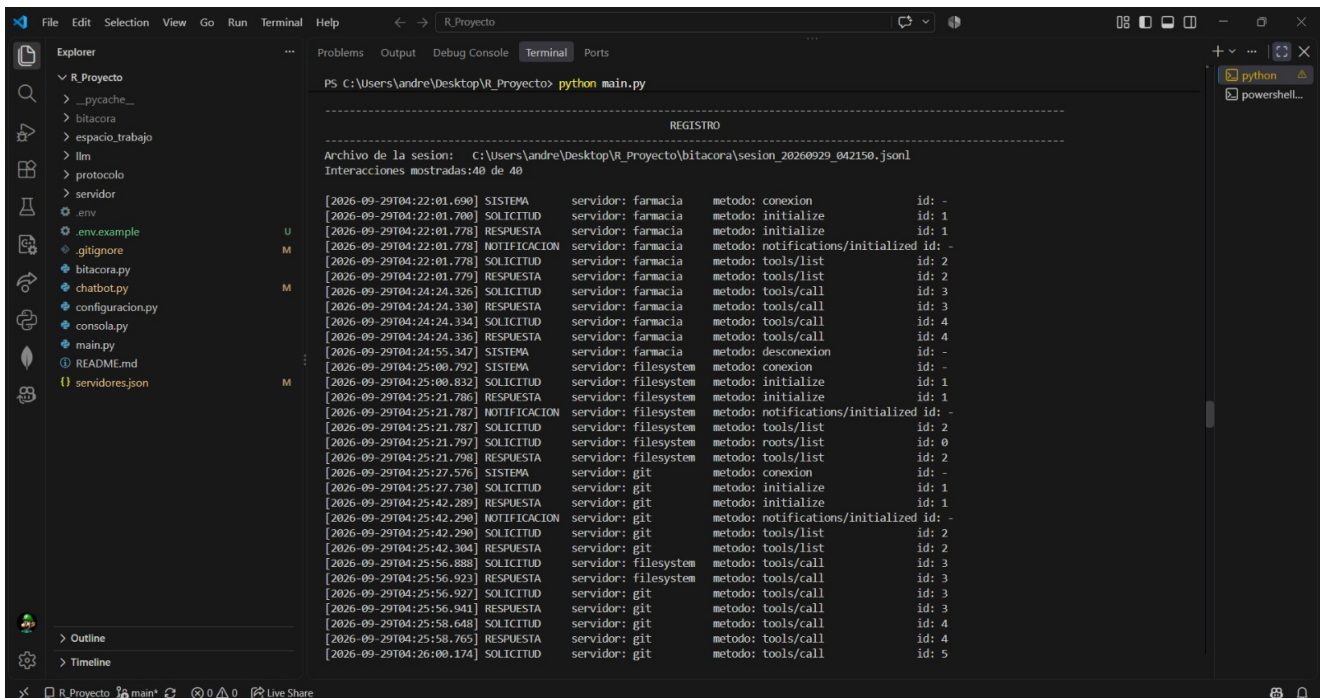


Figura 5: Resumen de la bitácora con las interacciones de los tres servidores locales

El resumen revela además un detalle del protocolo: inmediatamente después de la notificación de inicialización, el servidor de Filesystem envió por iniciativa propia la

solicitud `roots/list` con identificador 0, dirigida al cliente. JSON-RPC es bidireccional, y un servidor puede hacer peticiones al cliente del mismo modo que el cliente las hace al servidor. El anfitrión la registra en la bitácora y el servidor continúa operando sobre el directorio que recibió por línea de comandos. La Figura 6 muestra el contenido completo de los mensajes, en este caso la respuesta del servidor remoto a `initialize`.

```

PS C:\Users\andre\Desktop\R_Proyecto> python main.py
Ingresa la cantidad de interacciones a mostrar: 4

-----
REGISTER
-----
Archivo de la sesion:  C:\Users\andre\Desktop\R_Proyecto\bitacora\sesion_20260929_042150.jsonl
Interacciones mostradas: 4 de 40

[2026-09-29T04:27:10.905] RESPUESTA    servidor: remota    metodo: initialize    id: 1
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "tools": {
        "listChanged": false
      },
      "logging": {}
    },
    "serverInfo": {
      "name": "farmacia-mcp-uvg",
      "version": "1.0.0",
      "title": "Cadena de farmacias: orientacion farmaceutica y compra de medicamentos"
    },
    "instructions": "Servidor de una cadena de farmacias. Permite consultar la ficha de un cliente con sus alergi..."
  }
}

[2026-09-29T04:27:10.907] NOTIFICACION  servidor: remota    metodo: notifications/initialized id: -
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}

[2026-09-29T04:27:11.105] SOLICITUD    servidor: remota    metodo: tools/list    id: 2
{

```

Figura 6: Detalle de la bitácora con el contenido JSON de cada mensaje

6. Especificación del servidor propio

6.1. Identidad y capacidades

Atributo	Valor
Nombre	farmacia-mcp-uvg
Versión	1.0.0
Versiones del protocolo	2025-11-25, 2025-06-18, 2025-03-26, 2024-11-05
Versión preferida	2025-06-18
Capacidades declaradas	tools y logging
Transporte local	stdio, un mensaje JSON por línea
Transporte remoto	Streamable HTTP
Endpoint público	https://farmacia-mcp.onrender.com/mcp

Tabla 3: Identidad del servidor propio

Si el cliente propone una versión soportada, el servidor la acepta; de lo contrario responde con su versión preferida y deja al cliente la decisión de continuar.

6.2. Métodos implementados

Método	Comportamiento
<code>initialize</code>	Negocia la versión y devuelve capacidades, identidad e instrucciones
<code>notifications/initialized</code>	Marca la sesión como lista; no genera respuesta
<code>ping</code>	Devuelve un resultado vacío
<code>tools/list</code>	Devuelve el catálogo de diez herramientas
<code>tools/call</code>	Ejecuta una herramienta y devuelve su resultado
<code>resources/list</code>	Devuelve una lista vacía
<code>resources/templates/list</code>	Devuelve una lista vacía
<code>prompts/list</code>	Devuelve una lista vacía
<code>logging/setLevel</code>	Devuelve un resultado vacío
Cualquier otro	Error <code>-32601</code> , método no encontrado

Tabla 4: Métodos del protocolo atendidos por el servidor

6.3. Herramientas

Herramienta	Parámetros obligatorios	Parámetros opcionales
consultar_cliente	codigo_cliente	
evaluar_sintomas	sintomas	edad, alergias
buscar_medicamento		termino, solo_venta_libre, categoria
listar_sucursales		municipio, servicio_domicilio
consultar_inventario	codigo_medicamento	codigo_sucursal
verificar_interacciones	codigos_medicamentos	
crear_pedido	codigo_cliente, items	codigo_sucursal, tipo_entrega
consultar_pedido	codigo_pedido	
listar_pedidos		codigo_cliente, estado
actualizar_pedido	codigo_pedido, estado	nota

Tabla 5: Herramientas expuestas por el servidor propio

Los identificadores siguen formatos fijos: los clientes van de CLI-1001 a CLI-1005, las sucursales de SUC-01 a SUC-04, los productos de venta libre de MED-1001 a MED-1008, los de receta de MED-2001 a MED-2004, y los pedidos se numeran como PED-00001.

6.4. Endpoints y códigos HTTP

Petición	Respuesta
POST/mcp	200 con el mensaje JSON-RPC de respuesta; 202 sin cuerpo si el mensaje es una notificación; 400 si el cuerpo no es JSON válido
DELETE/mcp	204; elimina la sesión indicada en el encabezado <code>Mcp-Session-Id</code>
GET/mcp	405; el servidor no mantiene flujos iniciados por el servidor
GET/health	200 con texto plano; verificación de estado para la plataforma
Otra ruta	404

Tabla 6: Endpoints de la variante remota

El servidor genera un identificador de sesión al atender `initialize` y lo devuelve en el encabezado `Mcp-Session-Id`. El cliente lo adjunta en todas las peticiones siguientes, lo que resuelve la tensión entre un protocolo de sesión como MCP y un transporte sin estado como HTTP, del mismo modo que una cookie de sesión en la web.

6.5. Errores de protocolo y errores de negocio

El servidor distingue dos categorías de falla. Un parámetro obligatorio faltante o una herramienta inexistente producen un error de protocolo con el código `-32602`. Una violación de las reglas de negocio, en cambio, produce una respuesta exitosa cuyo resultado lleva la bandera `isError` en verdadero. La razón es práctica: un error de protocolo interrumpe el intercambio, mientras que un error de negocio es información que el modelo debe leer y explicar al usuario.

El Fragmento 3 muestra una invocación y la estructura de su respuesta.

Fragmento 3: Invocación de una herramienta y estructura de la respuesta

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "consultar_inventario",
    "arguments": {
      "codigo_medicamento": "MED-1001",
      "codigo_sucursal": "SUC-02"
    }
  }
}
```

```
"result": {"content": [{"type": "text", "text": "{ ... }"}], "isError": false
}
```

7. Despliegue del servidor remoto

La variante remota se desplegó en el plan gratuito de Render a partir del `Dockerfile` incluido en el repositorio. La Tabla 7 resume la configuración del servicio.

Parámetro	Valor
Tipo de servicio	Web Service
Directorio raíz	<code>servidor</code>
Entorno	Docker, imagen base <code>python:3.11-slim</code>
Plan	Free
Ruta de verificación	<code>/health</code>
Puerto	Asignado por la plataforma mediante la variable <code>PORT</code>
URL pública	<code>https://farmacia-mcp.onrender.com</code>

Tabla 7: Configuración del servicio en Render

El plan gratuito suspende el servicio tras quince minutos sin tráfico, y la primera petición posterior tarda cerca de un minuto en recibir respuesta mientras el contenedor vuelve a arrancar [4]. Por esa razón, antes de cada conexión se consultó la ruta de verificación, como muestra la Figura 7.

Para demostrar que el anfitrión utiliza el servidor remoto del mismo modo que el local, se desconectó el servidor local y se conectó únicamente el remoto. La Figura 8 muestra la conexión con las diez herramientas y la versión negociada.

La Figura 9 muestra la misma consulta de la sección anterior, ahora resuelta con la herramienta `remota_consultar_cliente`. El resultado es el mismo que en modo local: mismo mensaje, misma respuesta, distinto transporte.

Los registros del servicio confirman del lado del servidor la llegada de los mensajes. La Figura 10 muestra los registros filtrados por la palabra `metodo`: las tres líneas del saludo inicial y la invocación posterior comparten el mismo identificador de sesión.

La Figura 11 muestra los registros sin filtrar y revela dos observaciones. Las peticiones `GET/health` cada cinco segundos provienen de `10.236.26.236`, el monitoreo

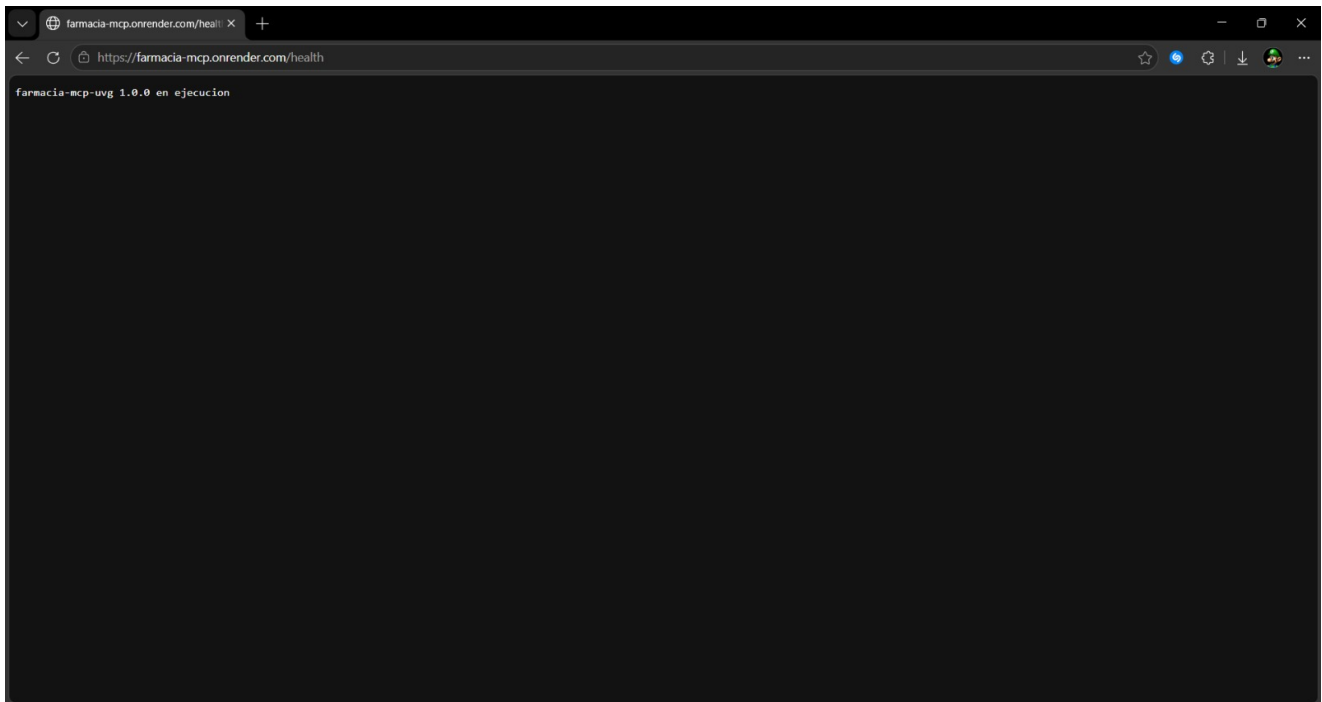


Figura 7: Verificación de estado del servidor desplegado

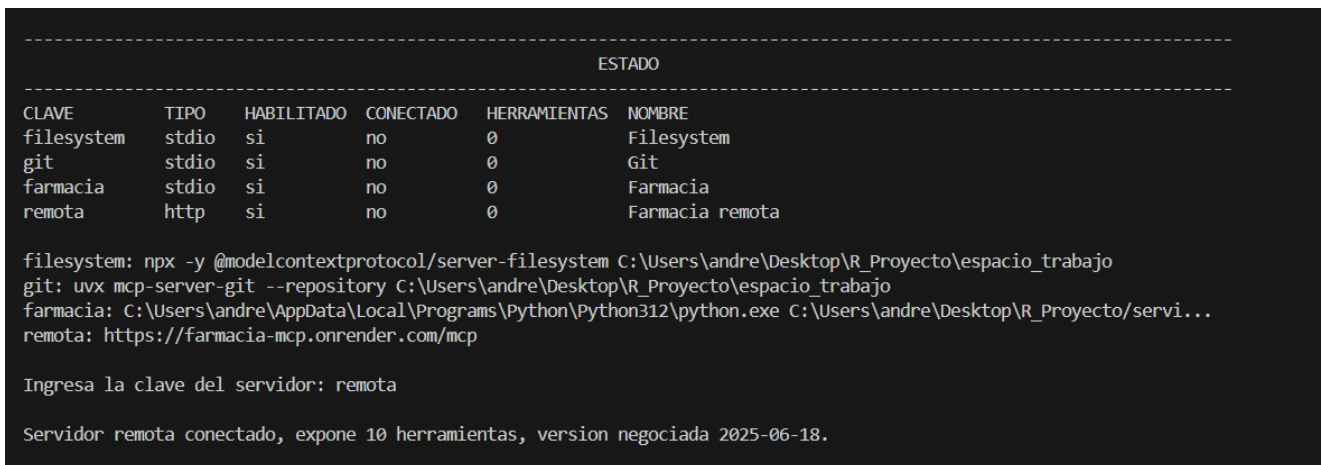


Figura 8: Conexión del anfitrión con el servidor remoto

interno de la plataforma. La petición `POST/mcp`, en cambio, llega al contenedor desde `127.0.0.1`, aunque salió desde Guatemala a través de Internet. Esto indica que la conexión TLS termina en un proxy de Render que se ejecuta junto al contenedor y le reenvía la petición en HTTP plano por la interfaz de loopback; por eso el servidor escucha en HTTP simple y aun así la dirección pública es HTTPS.

```

-----
COMANDOS
-----

Salir: regresa al menu principal
Limpiar: reinicia el contexto de la conversacion
Contexto: muestra la cantidad de mensajes almacenados
Herramientas: muestra la cantidad de herramientas disponibles

Consulta: Soy CLI-1001 y quiero comprar amoxicilina

Invocando herramienta: remota_consultar_cliente
Argumentos: {"codigo_cliente": "CLI-1001"}
Resultado de remota_consultar_cliente: correcto
Contenido: { "codigo": "CLI-1001", "nombre": "Maria Fernanda Ruiz", "edad": 34, "municipio": "Guatemala", "s...

Respuesta:
Voy a consultar tu perfil para verificar tus datos, alergias y recetas vigentes antes de procesar tu solicitud. No
puedo procesar tu solicitud de amoxicilina. Según tu perfil registrado, tienes alergia declarada a penicilina, y la
amoxicilina es un antibiótico de la familia de las penicilinas. Dispensar este medicamento representaría un riesgo grave
para tu salud. Recomendación: Consulta con un médico o farmacéutico en nuestra sucursal (Zona 10, disponible de lunes a
domingo 06:00-23:00) para que te prescriba una alternativa segura según tu condición.

Consulta: 

```

Figura 9: Consulta resuelta con el servidor remoto

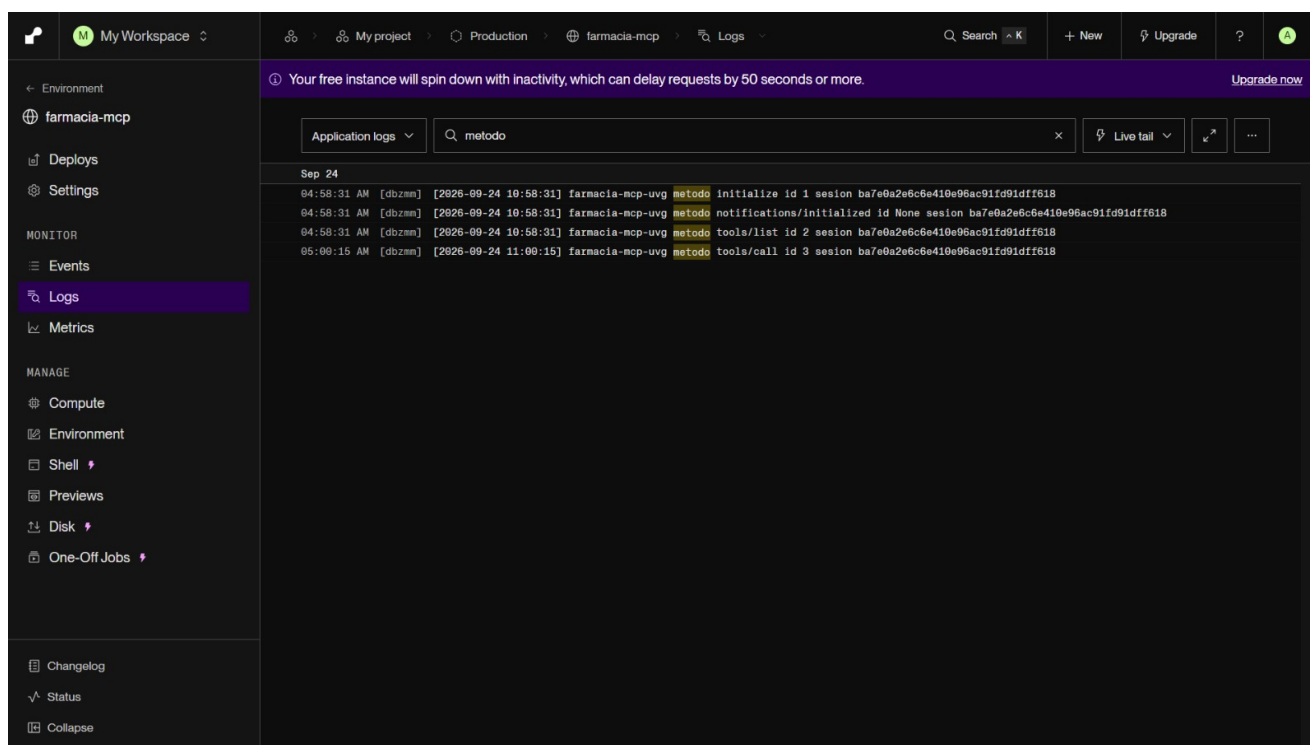


Figura 10: Registros del servidor en Render con los métodos recibidos

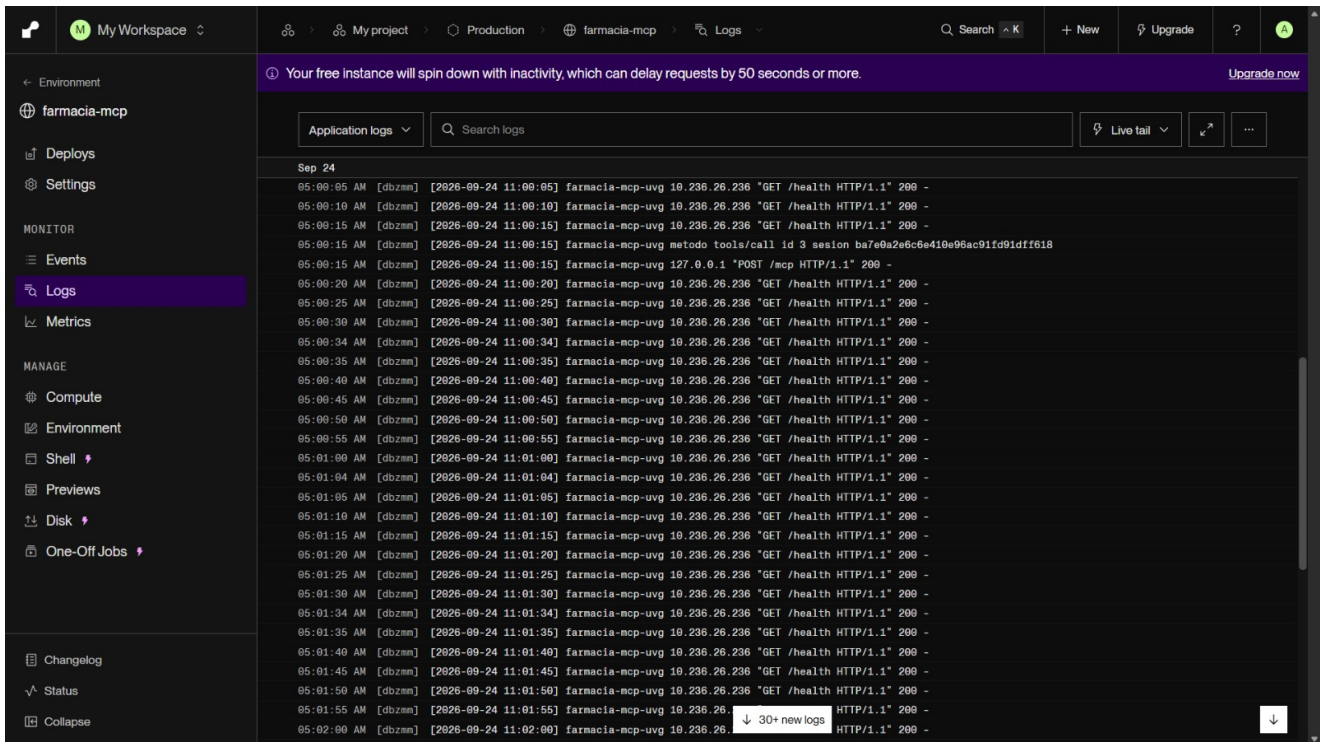


Figura 11: Registros sin filtrar: verificaciones de estado y petición reenviada por el proxy

8. Análisis de la comunicación

8.1. Metodología

La comunicación con el servidor remoto viaja cifrada con TLS, por lo que Wireshark solo mostraría registros de datos ilegibles. Para descifrarla se definió la variable de entorno `SSLKEYLOGFILE`, que hace que el módulo `ssl` de Python escriba las claves de sesión en un archivo, y ese mismo archivo se configuró en las preferencias del protocolo TLS de Wireshark [5]. Durante la captura no se utilizó el chat, para evitar que el tráfico hacia la API del modelo, que transporta la clave de acceso, quedara descifrado.

La captura se realizó sobre la interfaz inalámbrica y registró 1968 paquetes. La Tabla 8 resume la secuencia ejecutada desde el menú del anfitrión, diseñada para generar cada tipo de mensaje del protocolo.

Acción en el anfitrión	Mensajes generados
Conexión del servidor remoto	<code>initialize,</code> <code>notifications/initialized,</code> <code>tools/list</code>
Verificación de conexión	<code>ping</code>
Consulta de un cliente existente	<code>tools/call</code> exitoso
Pedido de un medicamento de receta	<code>tools/call</code> con <code>isError</code> en verdadero
Invocación sin argumentos obligatorios	<code>tools/call</code> con error <code>-32602</code>
Cierre del programa	<code>DELETE/mcp</code>

Tabla 8: Secuencia ejecutada durante la captura

8.2. Resolución del nombre del servidor

Antes de cualquier conexión, el sistema operativo resuelve el nombre `farmacia-mcp.onrender.com`. La Figura 12 muestra la respuesta a la consulta de tipo A, que devuelve una cadena de dos alias hacia un nombre de la red de distribución de Cloudflare y dos direcciones IPv4, `216.24.57.18` y `216.24.57.16`. Publicar varias direcciones para el mismo nombre permite que el cliente elija una y recurra a la otra si la primera falla.

La respuesta llegó en 4.66 milisegundos, un tiempo compatible con una respuesta almacenada en la caché del enrutador local. Las consultas de tipo AAAA solo devuelven los alias, sin ninguna dirección IPv6, por lo que la conexión posterior se realiza sobre IPv4. La consulta se repite antes de cada mensaje del protocolo, consecuencia directa de que el cliente abra una conexión nueva por cada petición.

8.3. Establecimiento de la conexión

La Figura 13 muestra, con el filtro por las dos direcciones del servidor, el ciclo completo de una petición. Los paquetes 677 a 679 forman el saludo de tres vías de TCP entre el puerto efímero 53074 y el puerto 443. El paquete 680 es el `ClientHello` de TLS 1.3, y los paquetes 682 y 683 completan el saludo del lado del servidor. El paquete 687 transporta la petición `POST/mcp` y el 691 su respuesta. Los paquetes 692 a 694 cierran la conexión, y en el paquete 697 inicia inmediatamente una conexión nueva desde el puerto 53075 para el siguiente mensaje.

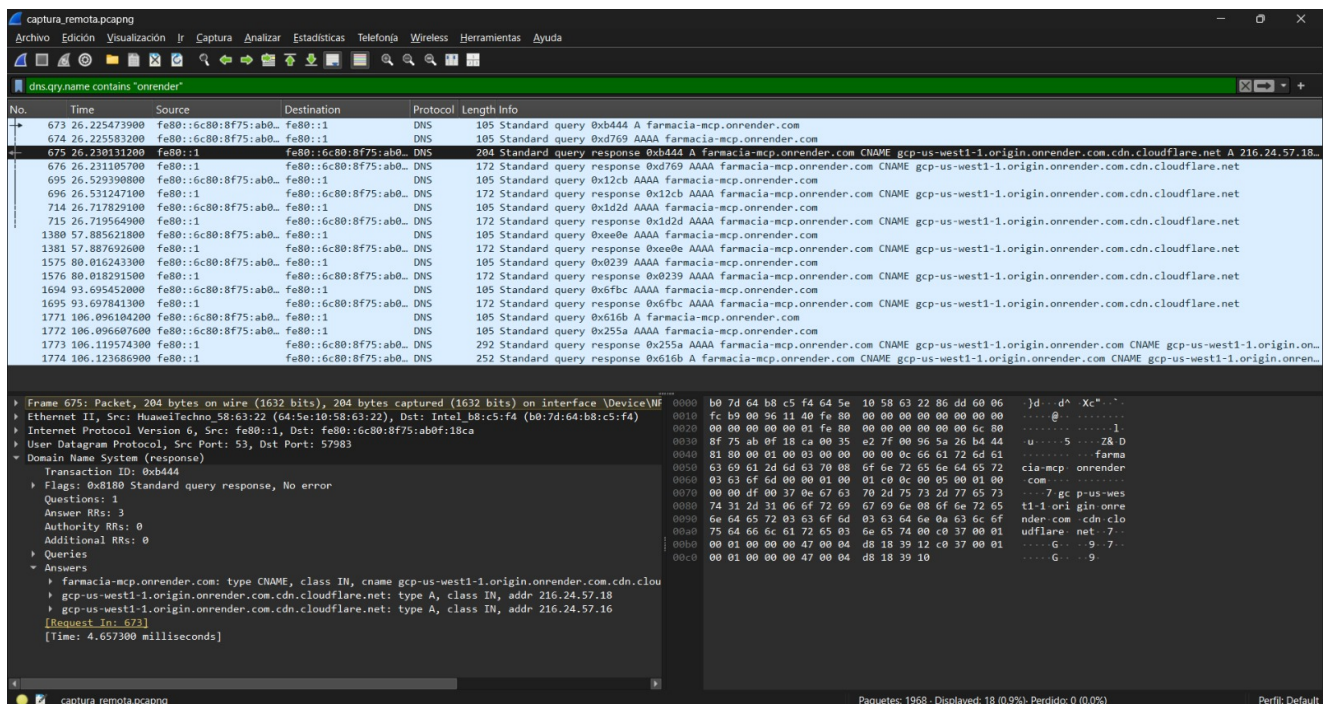


Figura 12: Resolución DNS del servidor remoto

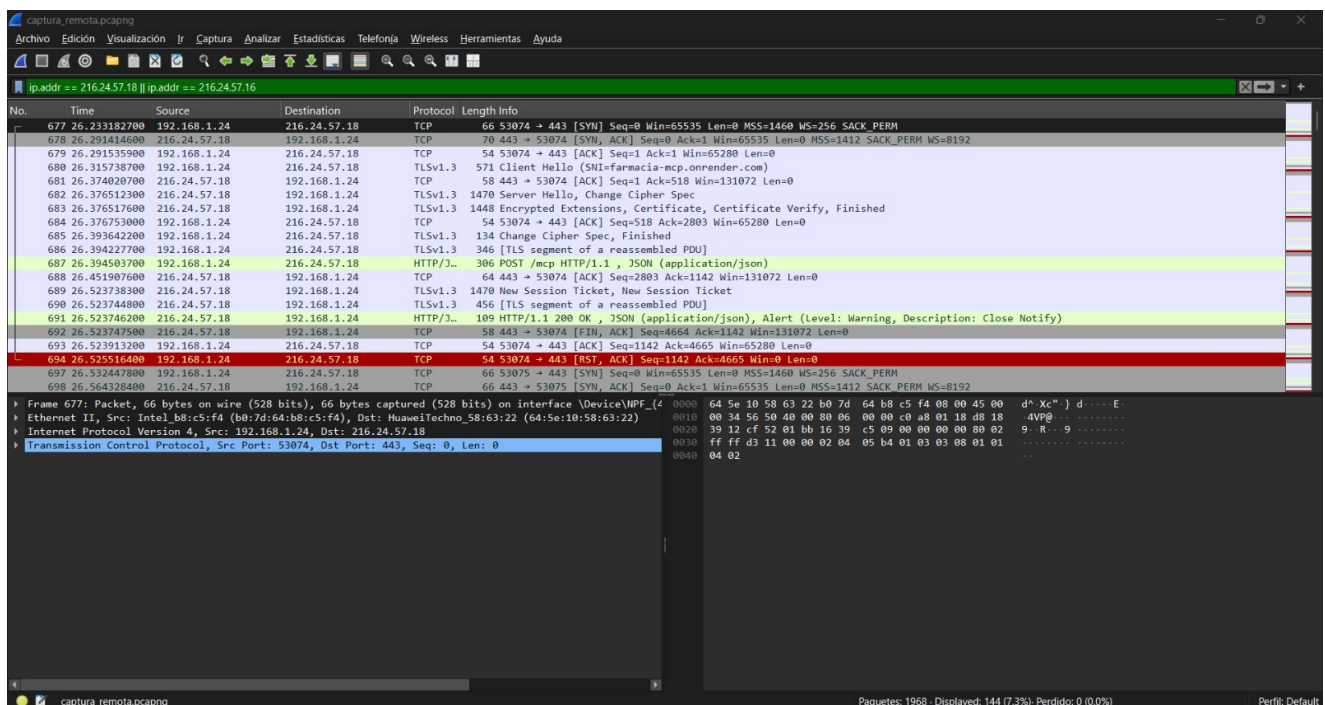


Figura 13: Saludo TCP, saludo TLS, intercambio y cierre de una petición

En el saludo TCP cada extremo anuncia su tamaño máximo de segmento: 1460 bytes el cliente, el valor típico para Ethernet, y 1412 bytes el servidor, un valor menor que deja espacio para encapsulamientos adicionales en la red del proveedor. Ambos extremos habilitan además el escalado de ventana y los acuses selectivos.

8.4. Mensajes del protocolo

Con el filtro `http` combinado con las direcciones del servidor, Wireshark muestra los dieciséis mensajes HTTP ya descifrados, que se observan en la Figura 14.

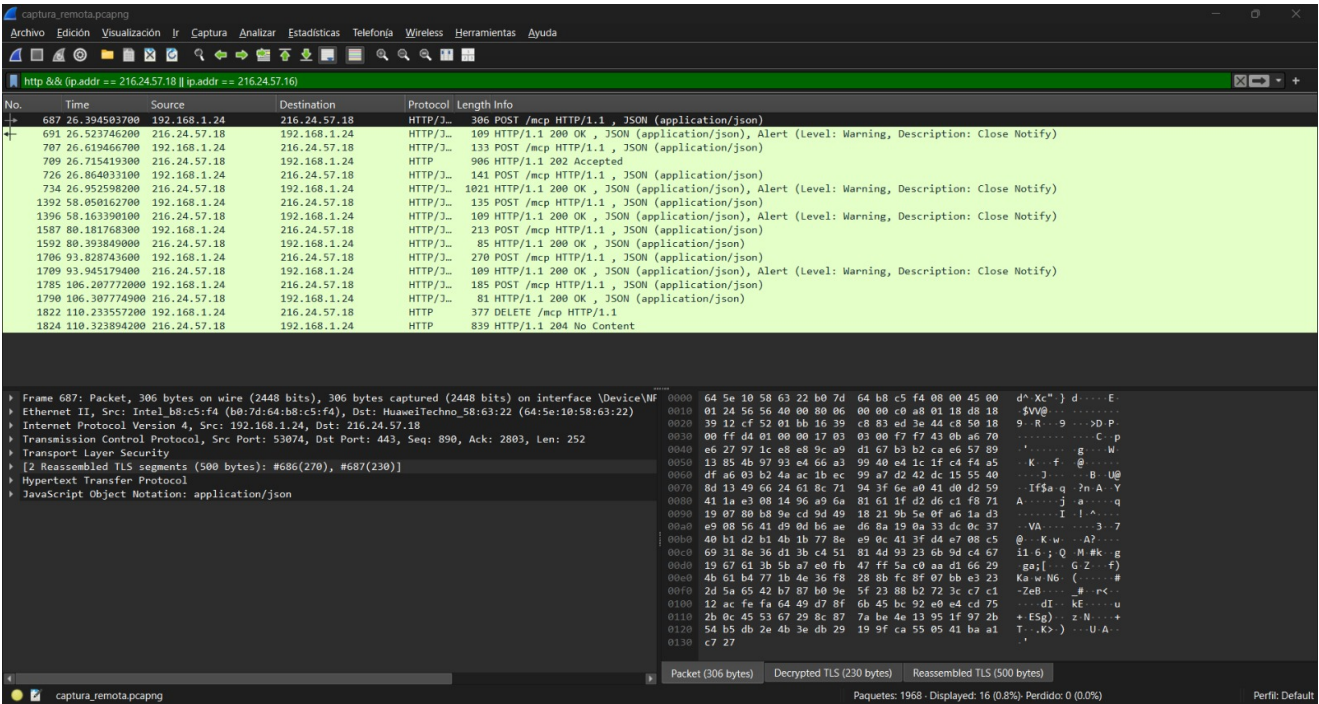


Figura 14: Mensajes HTTP descifrados entre el anfitrión y el servidor remoto

La Tabla 9 clasifica cada intercambio según la función que cumple en el protocolo.

Paquetes	Mensaje	Tipo JSON-RPC	Clase	HTTP
687 y 691	<code>initialize</code>	Solicitud y respuesta, id 1	Sincronización	200
707 y 709	<code>notifications/initialized</code>	Notificación sin id	Sincronización	202
726 y 734	<code>tools/list</code>	Solicitud y respuesta, id 2	Petición	200
1392 y 1396	<code>ping</code>	Solicitud y respuesta, id 3	Sincronización	200
1587 y 1592	<code>tools/call</code> exitoso	Solicitud y respuesta, id 4	Petición	200
1706 y 1709	<code>tools/call</code> con <code>isError</code>	Solicitud y respuesta, id 5	Petición	200
1785 y 1790	<code>tools/call</code> con error	Solicitud y error -32602, id 6	Petición	200
1822 y 1824	<code>DELETE/mcp</code>	No es JSON-RPC	Cierre	204

Tabla 9: Clasificación de los mensajes capturados

Los mensajes de sincronización son los que establecen, confirman o verifican el estado de la sesión sin ejecutar trabajo; las peticiones son las que solicitan el catálogo o la ejecución de una herramienta; y cada respuesta se reconoce por traer el mismo identificador que su solicitud. Dos detalles confirman el comportamiento especificado: la notificación es el único mensaje que el servidor responde con `202Accepted` y sin cuerpo, y el error de parámetros inválidos viaja con código HTTP 200, porque en este transporte el error pertenece a la capa JSON-RPC y no a la capa HTTP.

8.5. Sesión y encabezados

La Figura 15 muestra los encabezados de la notificación `notifications/initialized`, paquete 707. El cuerpo mide 57 bytes y la petición ya incluye el encabezado `Mcp-Session-Id` con el valor `34ff3daa640744c1b34dbc377abfbcf7`, que el servidor entregó en la respuesta a `initialize`. También se observan el encabezado `Mcp-Protocol-Version` con la versión negociada y el encabezado `Connection:close`.

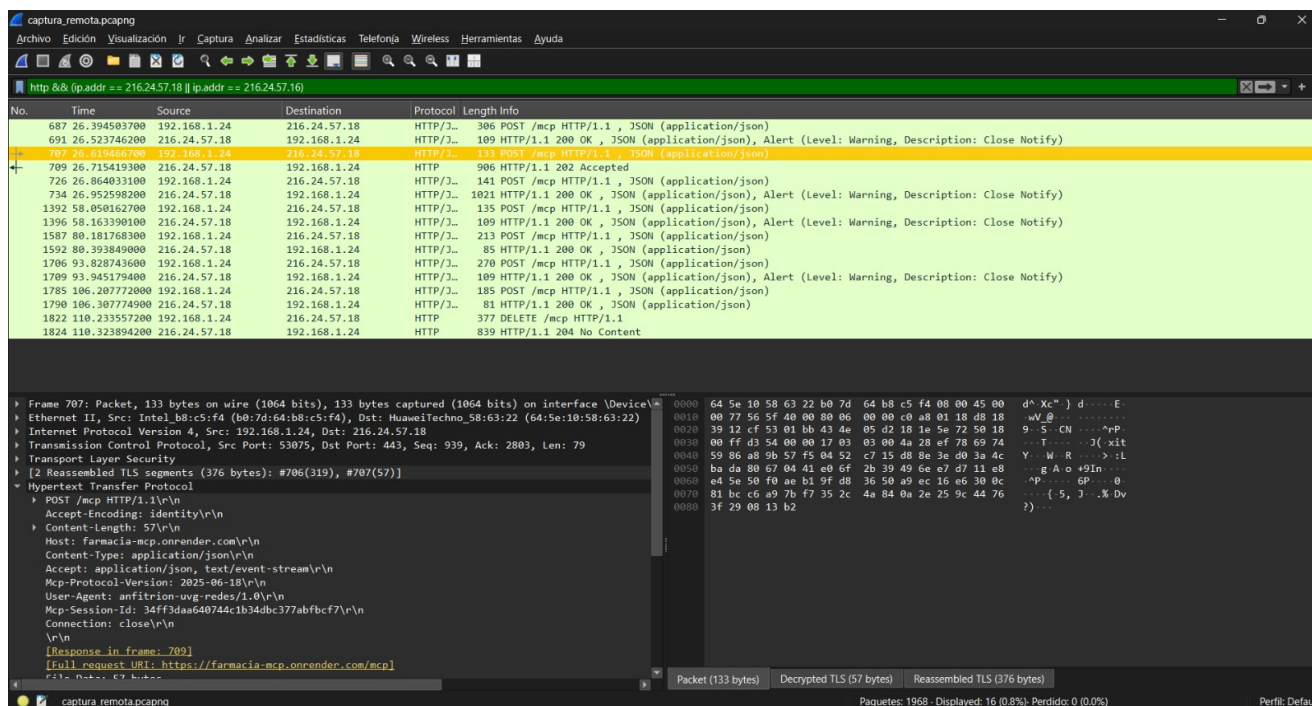


Figura 15: Encabezados HTTP de la notificación con el identificador de sesión

8.6. Contenido JSON-RPC

La Figura 16 muestra el paquete 726 con el mensaje JSON-RPC desplegado. Wireshark reconstruye el cuerpo a partir de dos registros TLS y lo interpreta como JSON: se observan el campo `jsonrpc` con valor 2.0 y el identificador 2, que corresponde a la solicitud `tools/list`.

8.7. Conversación completa

La Figura 17 reconstruye la conversación completa de esa petición: en rojo la solicitud del cliente y en azul la respuesta del servidor con el catálogo de herramientas.

Los encabezados de la respuesta revelan los intermediarios que existen entre el cliente y el contenedor. El encabezado `Server` indica `cloudflare`, mientras que `x-render-origin-server` identifica al servidor real como `farmacia-mcp-uvg/1.0.0` sobre Python 3.11. El servidor propio envía sus respuestas con `Content-Length`, pero el cliente las recibe con `Transfer-Encoding: chunked`, y el encabezado de sesión llega en minúsculas; ambas son señales de que un intermediario reconstruyó la respuesta antes de reenviarla. El sufijo del encabezado `CF-RAY` identifica el nodo de borde que atendió la petición, y el encabezado `alt-svc` anuncia que el mismo servicio está disponible sobre HTTP/3.

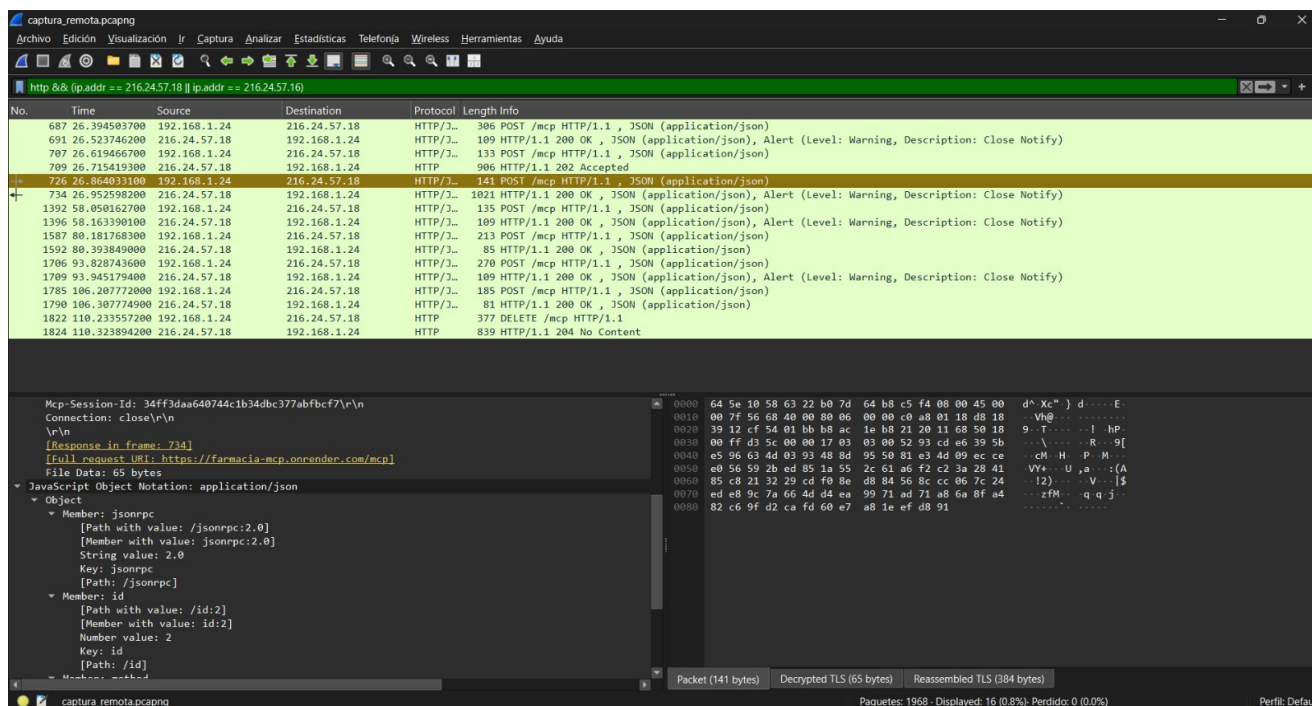


Figura 16: Contenido JSON-RPC de la solicitud tools/list

8.8. Estadística de la conversación

La Figura 18 muestra la estadística de la conexión TCP que transportó la solicitud `tools/list`, identificada como el flujo 37. La conexión intercambió 23 paquetes: 11 del cliente al servidor con cerca de 2 kB y 12 del servidor al cliente con cerca de 10 kB. La asimetría se explica por el tamaño del catálogo de herramientas frente al de la solicitud.

En total, los ocho intercambios de la sesión generaron 144 paquetes con el servidor, un promedio de dieciocho por mensaje. La mayor parte corresponde al establecimiento y cierre de conexiones TCP y TLS, no a los datos del protocolo, y es consecuencia de que el cliente abra una conexión nueva por cada mensaje.

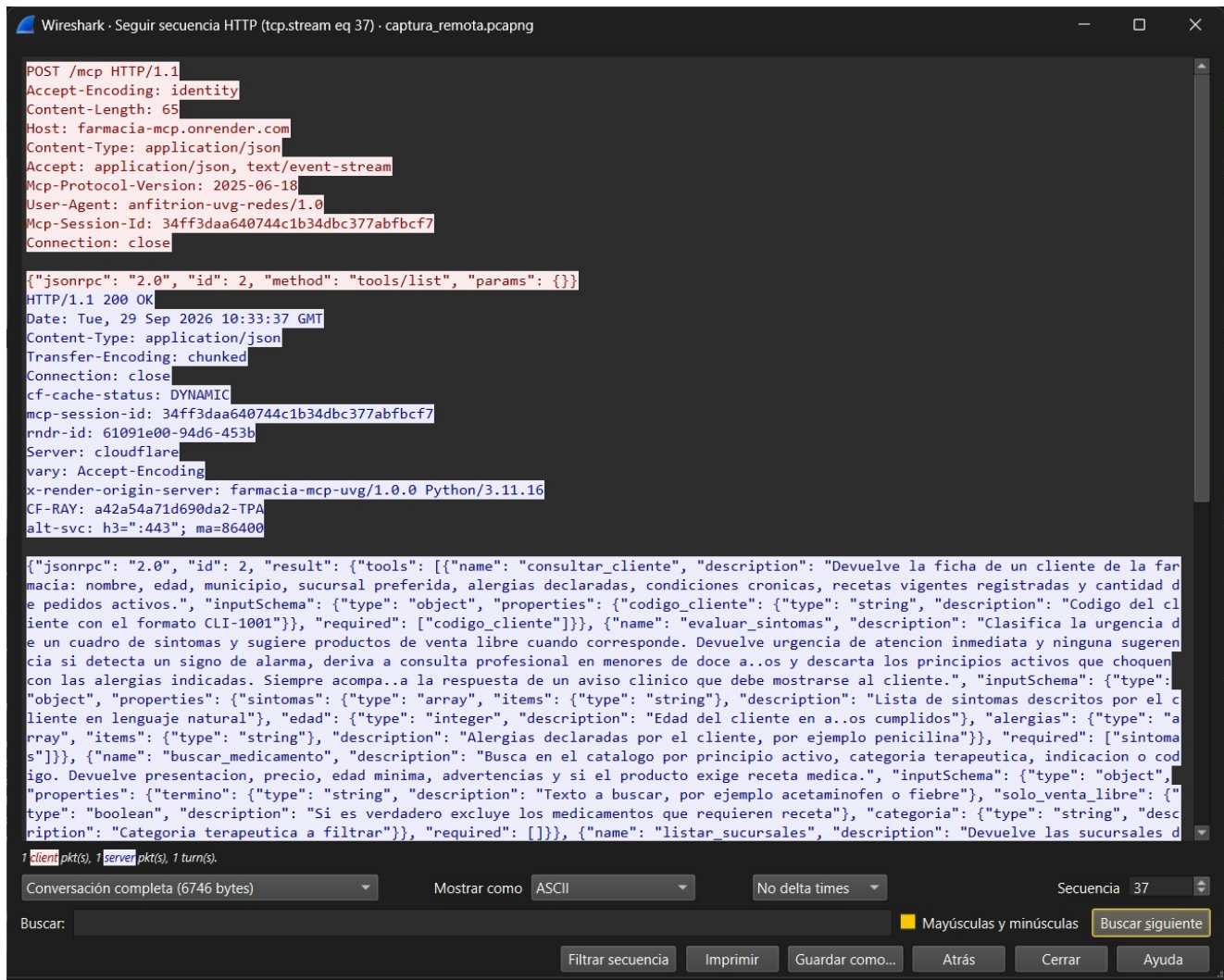


Figura 17: Secuencia HTTP completa de la solicitud tools/list

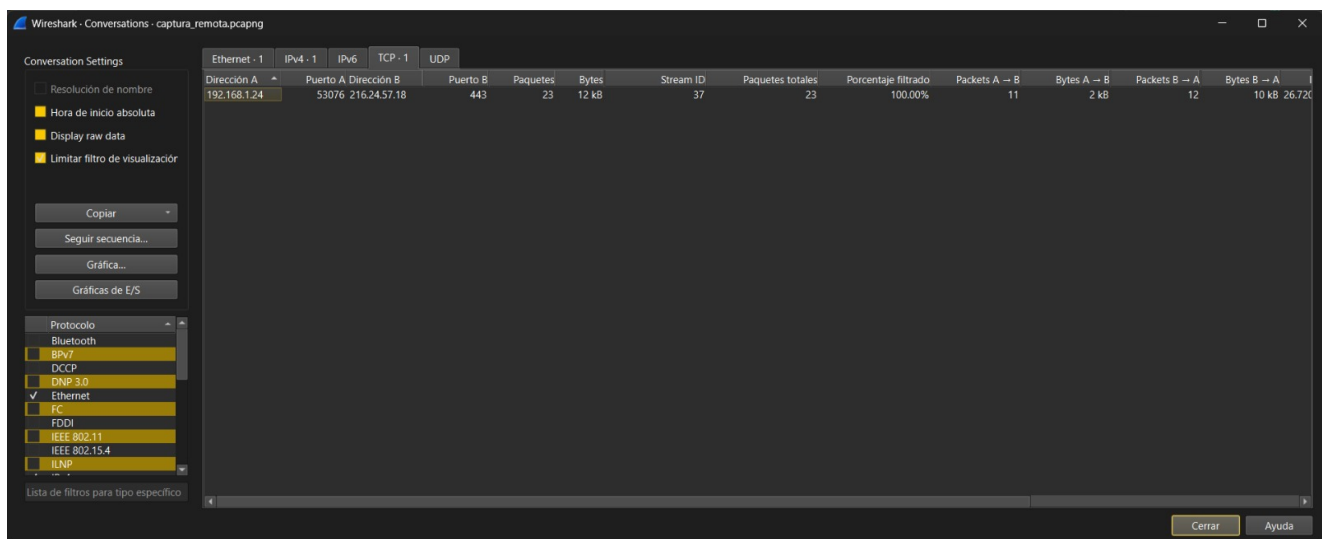


Figura 18: Estadística de la conexión TCP de la solicitud tools/list

9. Análisis por capas

La Tabla 10 resume lo observado en cada capa para el paquete 687, la solicitud `initialize`.

Capa	Protocolo	Observación
Enlace	Ethernet II	Origen <code>b0:7d:64:b8:c5:f4</code> , tarjeta Intel; destino <code>64:5e:10:58:63:22</code> , enrutador Huawei
Red	IPv4	Origen <code>192.168.1.24</code> , dirección privada; destino <code>216.24.57.18</code>
Transporte	TCP	Puerto efímero 53074 hacia el puerto 443
Seguridad	TLS 1.3	Cifrado de la sesión; nombre del servidor visible en el <code>ClientHello</code>
Aplicación	HTTP/1.1	<code>POST/mcp</code> con encabezados de sesión y versión del protocolo
Aplicación	JSON-RPC 2.0	Solicitud con <code>id 1</code> y método <code>initialize</code>
Aplicación	MCP	Negociación de versión y capacidades

Tabla 10: Encapsulamiento de la solicitud `initialize`

9.1. Capa de enlace

La trama Ethernet tiene como origen la tarjeta de red del equipo y como destino el enrutador local, no el servidor de Render. Las direcciones físicas solo tienen alcance en el primer salto: cada enrutador del camino reescribe la trama con sus propias direcciones. La respuesta DNS de la Figura 12 muestra la dirección inversa, con el enrutador como origen y la tarjeta como destino. Aunque la captura se realizó sobre una interfaz inalámbrica, el controlador de captura en Windows presenta las tramas como Ethernet II; el encabezado 802.11 original solo es visible en modo monitor.

9.2. Capa de red

El datagrama IPv4 lleva como origen la dirección privada `192.168.1.24`, que el enrutador traduce a su dirección pública mediante NAT antes de enviarla a Internet. El destino es una dirección de la red de Cloudflare y no del contenedor. La resolución DNS, en cambio, viajó sobre IPv6 con direcciones de enlace local `fe80::`, lo que

evidencia que la red local opera en doble pila aunque el servicio remoto solo se alcance por IPv4.

9.3. Capa de transporte

TCP aporta el establecimiento de la conexión, la entrega ordenada y la retransmisión. El puerto de origen es efímero y cambia en cada petición, 53074, 53075, 53076, mientras que el puerto de destino es siempre el 443. El cierre se inicia del lado del servidor con `FIN` y el cliente termina la conexión con `RST`, un comportamiento habitual cuando el socket se cierra inmediatamente después de leer la respuesta. La consulta DNS, por su parte, viajó sobre UDP hacia el puerto 53, sin establecimiento de conexión.

9.4. Seguridad y aplicación

TLS 1.3 se ubica entre el transporte y la aplicación y cifra todo el contenido HTTP. Aun así, el nombre del servidor viaja en claro dentro de la extensión `SNI` del `ClientHello`, porque el intermediario necesita conocerlo antes de establecer el cifrado para elegir el certificado correcto. Junto a cada respuesta el servidor envía la alerta `TLS close_notify`, que anuncia el cierre ordenado de la sesión cifrada, coherente con el encabezado `Connection:close`.

En la capa de aplicación conviven tres protocolos apilados: HTTP transporta en su cuerpo un mensaje JSON-RPC, que a su vez transporta un mensaje MCP. Esta superposición es la que se describió en la introducción.

9.5. Contraste con la captura de loopback

Para contrastar, se capturó el mismo servidor ejecutado localmente sobre HTTP en el puerto 8080, sobre la interfaz de loopback. La Figura 19 muestra el paquete 107, la solicitud `initialize` de esa captura.

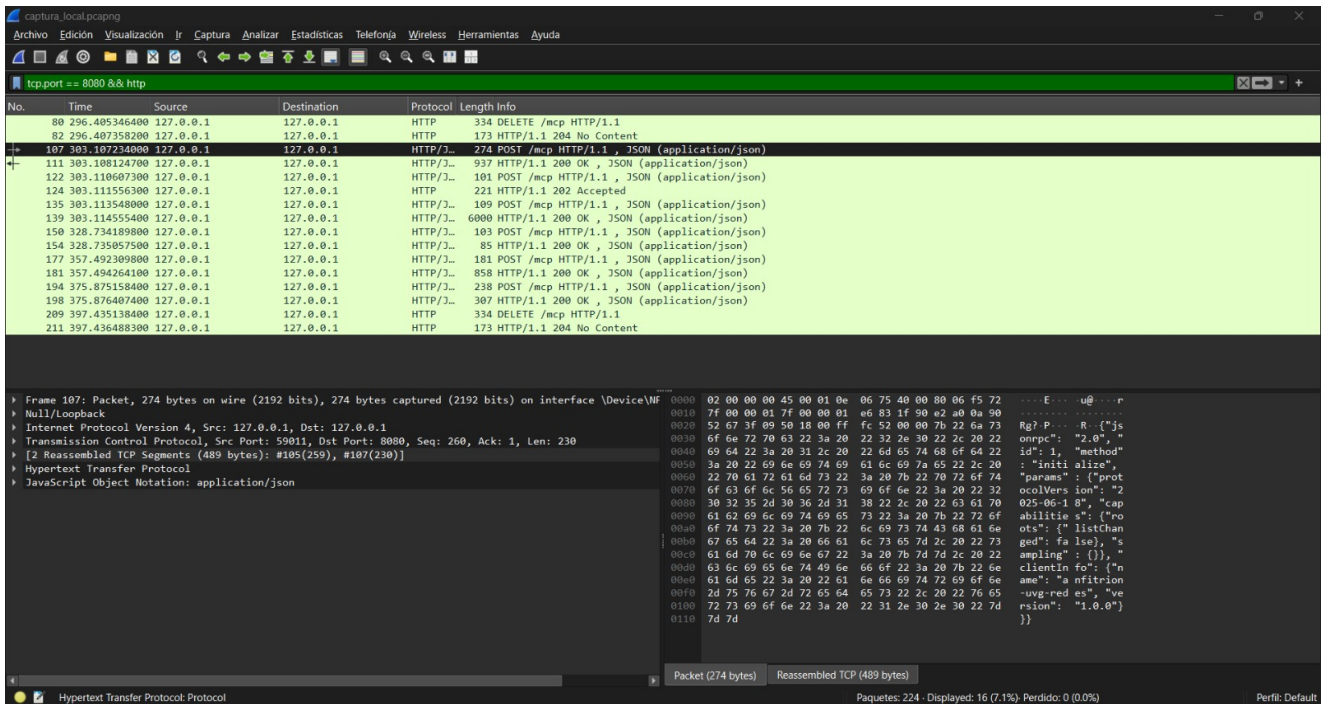


Figura 19: Solicitud initialize sobre la interfaz de loopback

Aspecto	Loopback	Remoto
Enlace	Null/Loopback, sin direcciones físicas	Ethernet II con direcciones físicas
Red	127.0.0.1 en ambos extremos	192.168.1.24 hacia 216.24.57.18
Transporte	Puerto 59011 hacia el 8080	Puerto 53074 hacia el 443
Seguridad	Sin cifrado	TLS 1.3
Petición	489 bytes en dos segmentos TCP de 259 y 230 bytes	500 bytes en dos registros TLS de 270 y 230 bytes
Cuerpo JSON-RPC	230 bytes	230 bytes

Tabla 11: Comparación de la solicitud initialize en ambas capturas

En la captura de loopback no existe la capa de enlace Ethernet: Wireshark muestra un encapsulamiento de loopback porque el paquete nunca sale a una red física, y tampoco existe TLS. La comparación de la Tabla 11 deja ver un resultado preciso: el cuerpo JSON-RPC mide exactamente 230 bytes en ambas capturas, y la diferencia de once bytes en los encabezados coincide con la diferencia de longitud del encabezado Host, que vale `farmacia-mcp.onrender.com` en un caso y `127.0.0.1:8080` en el otro. En ambos casos el cliente envía los encabezados y el cuerpo en dos escrituras

separadas, que Wireshark reensambla. El mensaje de aplicación es idéntico byte por byte; lo único que cambia es la pila que lo transporta.

10. Conclusiones

El protocolo de aplicación es independiente del medio que lo transporta. La misma solicitud `initialize` de 230 bytes cruzó una tubería del sistema operativo en modo `stdio`, la interfaz de loopback en modo HTTP local, y la pila completa de Ethernet, IPv4, TCP, TLS y HTTP en modo remoto, sin que el cliente ni el servidor modificaran su lógica. Esa independencia es lo que convierte a MCP en un estándar y no en una integración puntual.

La distinción entre errores de protocolo y errores de herramienta resultó fundamental en la práctica. Cuando el servidor de Git rechazó una ruta inválida, el error llegó como resultado de la herramienta y el modelo corrigió su propia invocación en el siguiente turno. De haberse tratado como error de protocolo, el intercambio se habría interrumpido.

Ubicar las reglas de negocio en el servidor, y no en el mensaje de sistema, es lo que hace confiable al sistema. El modelo puede razonar de forma incorrecta, pero no puede despachar un medicamento de receta si el servidor no lo permite.

Abrir una conexión por mensaje tiene un costo medible. Cada intercambio JSON-RPC implicó una consulta DNS, un saludo TCP y un saludo TLS completos, lo que produjo un promedio de dieciocho paquetes por mensaje. Reutilizar la conexión reduciría ese costo de forma considerable.

Entre el cliente y el servidor existen intermediarios que no son visibles desde el código. La captura y los registros de Render mostraron un nodo de borde de Cloudflare, un proxy que termina TLS y reenvía la petición por loopback, y la reescritura de encabezados y del formato de transferencia de la respuesta.

10.1. Dificultades y soluciones

- Un servidor que fallaba al arrancar solo reportaba que su salida se había cerrado. Se resolvió capturando la salida de error del proceso hijo y mostrándola junto al mensaje de falla, lo que permitió identificar causas como la ausencia de un repositorio en el directorio de trabajo.
- Los servidores pueden enviar mensajes por iniciativa propia en medio de un intercambio, como la solicitud `roots/list` observada en la bitácora. Se re-

solvió correlacionando cada respuesta por su identificador en lugar de leer el siguiente mensaje recibido.

- El plan gratuito de Render suspende el servicio tras un periodo de inactividad. Se resolvió consultando la ruta de verificación antes de cada conexión.
- El tráfico hacia el servidor remoto viaja cifrado. Se resolvió registrando las claves de sesión TLS y configurándolas en Wireshark, sin modificar el código.

10.2. Lecciones aprendidas

Implementar el protocolo de forma manual obligó a leer la especificación completa, y eso hizo visible el diseño en capas que un SDK oculta. Analizar el tráfico real mostró que un mensaje aparentemente simple atraviesa varios intermediarios, y que la eficiencia de un sistema distribuido depende tanto del protocolo de aplicación como de las decisiones sobre el transporte, como reutilizar o no una conexión. Finalmente, la separación de responsabilidades entre el modelo, que orquesta, y la herramienta, que decide, es la idea central que hace útil al protocolo.

Referencias

- [1] Model Context Protocol. *Specification, versión 2025-06-18*. <https://modelcontextprotocol.io/specification/2025-06-18>
- [2] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. <https://www.jsonrpc.org/specification>
- [3] Anthropic. *Messages API*. <https://docs.claude.com/en/api/messages>
- [4] Render. *Deploy for Free*. <https://render.com/docs/free>
- [5] Wireshark. *TLS*. <https://wiki.wireshark.org/TLS>
- [6] E. Rescorla. *RFC 8446: The Transport Layer Security Protocol Version 1.3*. IETF, 2018. <https://www.rfc-editor.org/rfc/rfc8446>
- [7] R. Fielding, M. Nottingham y J. Reschke. *RFC 9112: HTTP/1.1*. IETF, 2022. <https://www.rfc-editor.org/rfc/rfc9112>